

CSU33032 Project 1 – A Web Proxy Server

Lucia Brown – 22336688

My submission consists of three Python files – one containing the proxy server (proxy.py), and two test scripts which provide meaningful outputs. One test script deals with HTTP requests, using example.com as its input (http_test.py). The other test script deals with HTTPS requests, using google.com as its input (https_test.py).

Proxy.py Overview:

handle_client_request; This is a large function where most of the work occurs. It handles incoming client requests, forwards them to the target server and relays the response back. It times out the client socket after 25 seconds. I chose 25 seconds as my initial code with a 2 second timeout presented issues when I attempted to implement the caching portion of the requirements.

read_full_request; This function reads the full HTTP request from the client socket. It reads in the data chunk by chunk using the recv functionality. It ceases reading when there is no more data being received, or once full HTTP headers have been read.

handle_http; This function is invoked by handle_client_request when the port number is realised to not be 443, thus it is an unsecured connection. This function first searches the cache to see if the cache has expired for the provided host and port. If the cache has not yet expired, it can service the request using the cache. If the cache cannot be used to handle the request then fresh data is fetched.

handle_https; This function is invoked by handle_client_request when the port number is discovered to be equal to 443/there is a CONNECT method.

forward_data; This function is used to forward data between two sockets. Due to the cognitive complexity of the program, Python prompted me to split up larger functions into smaller ones, thus I had to create this function to reduce the load on my larger functions.

close_sockets; This function is used to safely close the sockets involved in the exchange.

url_is_blocked; This function is used during requests to validate if the requested URL is or is not blocked.

blocklist; This presents a menu to the user that allows them to add URLs to a blocklist, remove URLs from the blocklist, view the blocklist or exit.

extract_host_port_from_request; This function extracts the host and the port from the HTTP request. It is used by *handle_client_request* to ensure that the host and port are valid. And used by *url_is_blocked* to check that the URL is valid to be used.

http_test.py and https_test.py Overview:

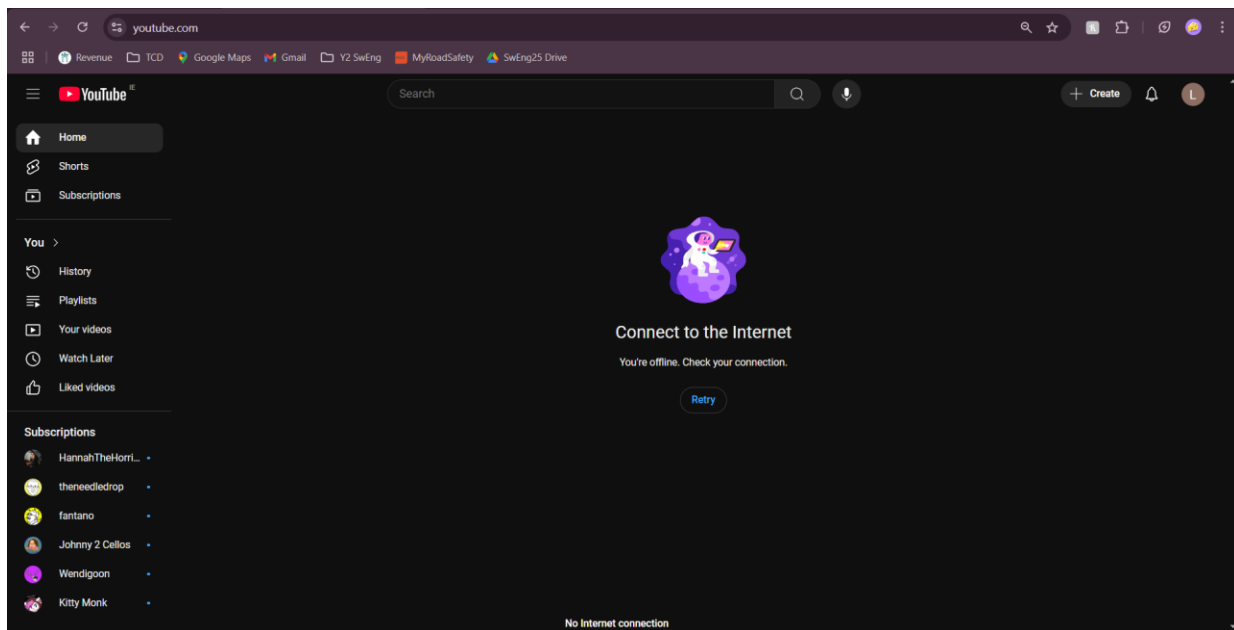
There are three tests in *http_test.py* and *https_test.py* respectively with the same test cases: the first test simply conducts one basic connection with the proxy server. It prints the HTTP response code, which should be 200, it prints the response header and the first 200 characters of the response body to ensure that everything is successful.

The second test ensures that the caching functionality works. This test works by making a request to the proxy. After a short propagation delay, a second identical request is made to the server and *response1* is compared to *response2*. If *response1* and *response2* are identical, it can be inferred that a cache hit was likely. After a larger delay, past the cache expiry time, a third request is sent. Thus, *response3* should take significantly longer to return than *response2*. Timing data is included in this test for comparison.

The third test ensures that the concurrency functionality works. This test simulates multiple users (3) requesting the same URL. This test verifies that all of the 'users' were able to access the server and that they all received the same responses from the server.

In-Browser Testing:

To test the efficiency of the URL blocking feature, with my proxy server running, I used the management console to block the www.youtube.com URL. This successfully prevented me from accessing the site:



Code:

proxy.py:

```
import socket, threading
from datetime import datetime, timedelta

# URL Blocking Initialisation & Cache
blocked_urls = set()
cache={}
cache_expiry=timedelta(seconds=30)
blocklist_lock = threading.Lock()

# Initialize socket server & bind to local address, listen for up to 10 incoming
connections
port = 8888
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', port))
server.listen(10)
print(f"Proxy server started on 127.0.0.1:{port}")

# Read data from incoming request & extract target server's host & port
def handle_client_request(client_socket):
    """Handles incoming client requests, forwards them to the target server, and
    relays the response back."""
    print("Received request from client.")
    client_socket.settimeout(25.0) # Set timeout for receiving
```

```

# Kill connection if there's no data found
request = read_full_request(client_socket)
if not request:
    print("No request data received. Closing connection.")
    client_socket.close()
    return

# Take the first line to extract the method (check for HTTPS)
first_line = request.split(b'\n')[0].decode('utf-8', 'ignore')
method, _, _ = first_line.split()

# Extract the target server's host and port from the request
host, port = extract_host_port_from_request(request)

# Kill connection if there's no host or port found
if not host or port is None:
    print("[Invalid host/port extracted. Closing connection.]")
    client_socket.close()
    return

print(f"Extracted Host: {host}, Port: {port}")

# Check if the requested URL is blocked
if url_is_blocked(host):
    print(f"URL {host} is blocked.")
    client_socket.sendall(b"HTTP/1.1 403 Forbidden\r\n\r\nBlocked URL.")
    client_socket.close()
    blocklist()
    return

# Check if the connection is HTTPS
if method == "CONNECT":
    handle_https(client_socket, host, port)
# Use HTTP instead of HTTPS
else:
    handle_http(client_socket, host, port, request)

def read_full_request(client_socket):
    """Reads the full HTTP request from the client socket."""
    request = b""
    try:
        while True:

```

```

        data = client_socket.recv(1024) # Take in data chunk by chunk
        if not data:
            break # No more data means request is complete
        request += data # Add the chunks to the full request
        if b'\r\n\r\n' in request: # Stop reading once full HTTP headers are
received
            break
    except socket.timeout:
        print("Client socket timed out while reading request (likely incomplete
request).")
    except Exception as e:
        print(f"Error while reading request: {e}")
    return request

# HTTP handler
def handle_http(client_socket, host, port, request):

    # If there is an existing cache, attempt to use it
    cache_key = f"{host}:{port}"
    if cache_key in cache:
        cache_timestamp, cached_response = cache[cache_key]
        if datetime.now() - cache_timestamp < cache_expiry: # Ensure that the
cache is still within the appropriate time
            print(f"Serving {cache_key} from cache.")
            try:
                client_socket.sendall(cached_response) # Ensure full response is
sent
            except Exception as e:
                print(f"Error sending cached response: {e}")
            client_socket.close()
            return
        else:
            print(f"Cache expired for {cache_key}, fetching fresh data.") # Get
new data if the cache is expired
            del cache[cache_key] # Clear old cache

    # Forward client request & handle response
    try:
        destination_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        destination_socket.connect((host, port))
        print("Connection established with target server.")

```

```

        # Forward request to destination server
        request = request.replace(b"Connection: keep-alive", b"Connection:
close")
        destination_socket.sendall(request)
        print("Request forwarded to target server.")

        response_data = b''
        while True:
            data = destination_socket.recv(1024)    # Handle data in chunks again
            if not data:
                break # Stop when no more data is available
            print(f"Received chunk from target server:")
            response_data += data # Collect full response
            client_socket.sendall(data)
        if response_data:
            cache[cache_key] = (datetime.now(), response_data) # Update cache
with response_data
            print(f"Cached response for {cache_key}")

    except Exception as e:
        print(f"Error forwarding request or receiving response: {e}")

    finally:
        close_sockets(client_socket, destination_socket)    # Terminate the
connection once all is complete

# HTTPS handler
def handle_https(client_socket, host, port):
    try:
        # Establish the HTTPS connection
        print(f"Setting up tunnel for {host}:{port}...")

        # Connect to target server
        destination_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        destination_socket.connect((host, port))

        # Respond to client indicating tunnel is established
        client_socket.sendall(b"HTTP/1.1 200 Connection Established\r\n\r\n")
        print("200 OK response sent to client.")

        # Use threading to allow bidirectional communication
        client_to_server = threading.Thread(target=forward_data,
args=(client_socket, destination_socket, "Client to Server Direction"))

```

```

        server_to_client = threading.Thread(target=forward_data,
args=(destination_socket, client_socket, "Server to Client Direction"))
        client_to_server.start()
        server_to_client.start()
        client_to_server.join(timeout=25)
        server_to_client.join(timeout=25)

    # Kill connection if still alive after the timeout
    if client_to_server.is_alive() or server_to_client.is_alive():
        print("Forcing socket closure due to prolonged connection.")
        close_sockets(client_socket, destination_socket)

except Exception as e:
    print(f"Error in HTTPS: {e}")

finally:
    close_sockets(client_socket, destination_socket)    # Terminate the
connection once all is complete

def forward_data(source, destination, direction):
    """Forwards data between two sockets and logs activity"""
    source.settimeout(10)
    try:
        while True:
            try:
                data = source.recv(4096)
                if not data:
                    print(f"[{direction}] Connection closed.")
                    break
                print(f"[{direction}] Forwarding {len(data)} bytes")
                destination.sendall(data)
            except socket.timeout:
                print(f"[{direction}] Socket timeout reached, closing
connection.")
                break
    except Exception as e:
        print(f"Error forwarding data: {e}")

# Close the connections/sockets
def close_sockets(client_socket, destination_socket):
    """Safely closes client and destination sockets."""
    print("Closing connections...")

```

```

# If there is a destination socket, attempt to close it
if destination_socket:
    try:
        destination_socket.close()
    except Exception as e:
        print(f"Error closing destination socket: {e}")

# If there is a client socket, attempt to close it
if client_socket:
    try:
        client_socket.close()
    except Exception as e:
        print(f"Error closing client socket: {e}")
print("Connection closed.")
blocklist()

# Checks for URL blockage
def url_is_blocked(host):
    """Checks if the requested host is in the blocklist."""
    for blocked in blocked_urls:
        if host in blocked or host.endswith("." + blocked):
            return True
    return False

# Allow for dynamic URL blocking
def blocklist():
    """Manage blocked URLs."""
    while True:
        print("1. Add URL to blocklist")
        print("2. Remove URL from blocklist")
        print("3. Show blocked URLs")
        print("4. Exit")
        choice = input("---- Enter your choice: ----")

        with blocklist_lock:
            if choice == '1':
                url = input("Enter URL to block: ")
                if url in blocked_urls:
                    print(f"{url} is already in the blocklist.")
                else:
                    blocked_urls.add(url)
                    print(f"{url} Added to blocklist.")
            continue

```

```

        elif choice == '2':
            url = input("Enter URL to unblock: ")
            if url not in blocked_urls:
                print(f"{url} is not in the blocklist.")
            else:
                blocked_urls.discard(url)
                print(f"{url} Removed from blocklist.")
            continue
        elif choice == '3':
            print("\n---- Blocked URLs: ----")
            if blocked_urls:
                for url in blocked_urls:
                    print(f" - {url}\n")
            else:
                print("No URLs are blocked.\n")
            continue
        elif choice == '4':
            print("Resuming...")
            break
        else:
            print("Invalid choice, please try again.")
            continue

# Get the host and port names
def extract_host_port_from_request(request):
    """Extracts the host and port from the HTTP request."""
    try:
        host_string_start = request.find(b'Host: ') + len(b'Host: ')
        host_string_end = request.find(b'\r\n', host_string_start)
        host_string = request[host_string_start:host_string_end].decode('utf-8')

        if ":" in host_string:
            host, port = host_string.split(":", 1)
            port = int(port)
        else:
            host = host_string
            port = 443 if b'CONNECT' in request.split(b'\n')[0] else 80 # HTTPS
            default to 443, HTTP to 80
        return host, port
    except Exception as e:
        print(f"Failed to extract host/port: {e}")
        return None, None

```

```

blocklist_thread = threading.Thread(target=blocklist)
blocklist_thread.start()

# Always listen for incoming requests & create a new thread to handle each client
request
if __name__ == "__main__":
    while True:
        client_socket, addr = server.accept()
        print(f"Accepted Connection from {addr[0]}:{addr[1]}")

        # Create a new thread to handle the client's request
        client_handler = threading.Thread(target=handle_client_request,
args=(client_socket,))
        client_handler.start()

```

http_test.py:

```

import requests,time,concurrent.futures

# Proxy settings
proxies = {
    "http": "http://127.0.0.1:8888",
    "https": "http://127.0.0.1:8888",
}

# Test sending a basic HTTP request through the server
def send_request(url):
    print("---- TEST 1 - BASIC HTTP CONNECTION ----")
    try:
        print(f"Sending request to {url} through the proxy...")

```

```

        response = requests.get(url, proxies=proxies)
        print("Response status code:", response.status_code) # Print HTTP status
code
        print("Response headers:", response.headers)
        print("Response body (first 200 chars):", response.text[:200]) # Print
first 200 chars of body
        return response
    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")
        return None

# Test the caching functionality by sending the same requests through the server
before and after the cache has expired
# Includes timing data to prove that the cached result is faster than the non-
cached result
def caching_test(response1):
    print("---- TEST 2 - CACHING CONNECTIONS ----")
    time.sleep(2)
    start_time_1 = time.time()
    response2 = send_request(url)
    end_time_1 = time.time()
    elapsed_time_1 = end_time_1 - start_time_1
    print(f"Time taken for cached request: {elapsed_time_1:.2f} seconds")

    # Compare results
    if response1 and response2:
        if response1.status_code == response2.status_code:
            print("Both requests returned the same status code.")
        if response1.text[:200] == response2.text[:200]:
            print("Responses have identical content (cache hit likely).")
        else:
            print("Responses have different content (cache miss or different
resource).")
    else:
        print("Issue with one of the requests.")

    time.sleep(40) # Allow the cache to expire
    start_time_2 = time.time()
    send_request(url)
    end_time_2 = time.time()
    elapsed_time_2 = end_time_2 - start_time_2
    print(f"Time taken for non-cached request: {elapsed_time_2:.2f} seconds")

```

```

        difference = elapsed_time_2 - elapsed_time_1
        print(f"THE TIME DIFFERENCE BETWEEN THE CACHED AND NON-CACHED REQUEST WAS:
{difference:.5f} seconds")
        print("---- END TEST 2 ----")

# Test for concurrent requests
def concurrent_test():
    print("---- TEST 3 - CONCURRENT CONNECTIONS ----")
    print("\nSending concurrent requests...\n")
    urls = [url] * 3 # Simulate multiple users requesting the same URL
    start_time = time.time()

    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        results = list(executor.map(send_request, urls))

    # Time the concurrent requests
    elapsed_time = time.time() - start_time
    print(f"\nAll concurrent requests completed in {elapsed_time:.6f} seconds")

    # Check if all responses are the same (cache verification)
    successful_responses = [resp for resp in results if resp and resp.status_code
== 200]
    if len(successful_responses) == len(urls):
        print("All concurrent requests returned successful responses")
        if all(resp.text[:200] == successful_responses[0].text[:200] for resp in
successful_responses):
            print("All concurrent responses are identical")
        else:
            print("Concurrent responses differ")
    else:
        print("Some concurrent requests failed")
    print("---- END TEST 3 ----")

url = "http://example.com"
response1 = send_request(url)
caching_test(response1)
concurrent_test()

```

https_test.py:

```
import requests, time, concurrent.futures
```

```

# Proxy settings (For HTTPS requests)
proxies = {
    "http": "http://127.0.0.1:8888", # Proxy server running locally
    "https": "http://127.0.0.1:8888",
}

# Send a basic HTTPS request through the proxy
def send_https_request(url):
    print("---- TEST 1 - BASIC HTTPS CONNECTION ----")
    try:
        print(f"Sending HTTPS request to {url} through the proxy...")
        response = requests.get(url, proxies=proxies, verify=False) # Disable
verification for testing
        print("Response status code:", response.status_code)
        print("Response headers:", str(response.headers)[:20])
        print("Response body (first 200 chars):", response.text[:200])
        return response
    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")
        return None

# Test caching functionality for HTTPS connections
# Includes timing data to prove cached request is faster than non-cached request
def caching_test_https(url):
    print("\n---- TEST 2 - CACHING HTTPS CONNECTIONS ----\n")
    response1 = send_https_request(url) #Initial request to ensure there is
something in the cache

    time.sleep(2)
    start_time_cache = time.time()
    response2 = send_https_request(url)    #This request should be pulled from
the cache
    end_time_cache = time.time()
    elapsed_time_cache = end_time_cache - start_time_cache
    print(f"Time taken for cached request: {elapsed_time_cache:.5f} seconds")

# Compare results
if response1 and response2:
    if response1.status_code == response2.status_code:
        print("Both HTTPS requests returned the same status code.")
    if response1.text[:200] == response2.text[:200]:
        print("HTTPS responses have identical content (cache hit likely).")
    else:

```

```

        print("HTTPS responses have different content (cache miss or
different resource).")
    else:
        print("Issue with one of the HTTPS requests.")

    time.sleep(40) # Allow the cache to expire
    start_time_nocache = time.time()
    send_https_request(url) # This request should be fresh and take longer
    end_time_nocache = time.time()
    elapsed_time_nocache = end_time_nocache - start_time_nocache
    print(f"Time taken for non-cached request: {elapsed_time_nocache:.5f}
seconds")

    difference = elapsed_time_nocache - elapsed_time_cache
    print(f"THE TIME DIFFERENCE BETWEEN THE CACHED AND NON-CACHED REQUEST WAS:
{difference:.5f} seconds")

    print("\n---- END TEST 2 ----\n")

# Test for concurrent HTTPS requests
def concurrent_https_test(url):
    print("---- TEST 3 - CONCURRENT HTTPS CONNECTIONS ----")
    print("\nSending concurrent HTTPS requests...\n")
    urls = [url] * 3 # Simulate multiple users requesting the same HTTPS URL
    start_time = time.time()

    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        results = list(executor.map(send_https_request, urls))

    # Time the concurrent requests
    elapsed_time = time.time() - start_time
    print(f"\nAll concurrent HTTPS requests completed in {elapsed_time:.6f}
seconds")

    # Check if all responses are the same
    successful_responses = [resp for resp in results if resp and resp.status_code
== 200]
    if len(successful_responses) == len(urls):
        print("All concurrent HTTPS requests returned successful responses")
        if all(resp.text[:200] == successful_responses[0].text[:200] for resp in
successful_responses):
            print("All concurrent HTTPS responses are identical")
        else:

```

```
        print("Concurrent HTTPS responses differ")
    else:
        print("Some concurrent HTTPS requests failed")
    print("---- END TEST 3 ----")

# Test the HTTPS functionality
google = "https://google.com"
youtube = "https://youtube.com"
github = "https://github.com"

send_https_request(youtube)
caching_test_https(google)
concurrent_https_test(github)
```